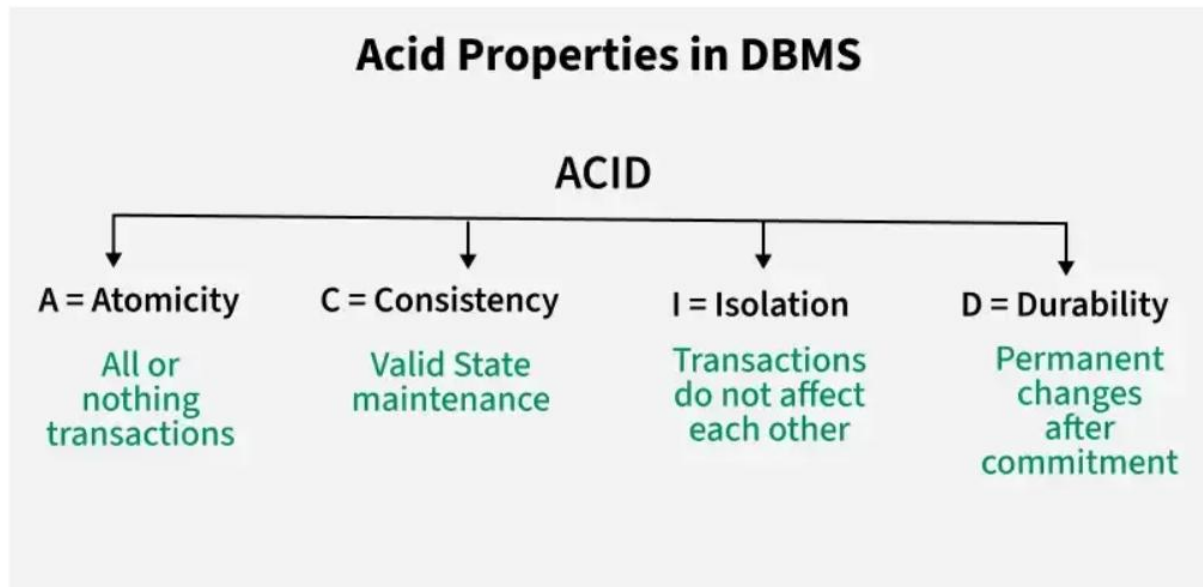


Transactions are fundamental operations that allow us to modify and retrieve data. However, to ensure the integrity of a database, it is important that these transactions are executed in a way that maintains consistency, correctness, and reliability even in the case of failures/errors. This is where the ACID properties come into play.



1. Atomicity

Atomicity means a transaction is all-or-nothing either all its operations succeed, or none are applied. If any part fails, the entire transaction is rolled back to keep the database consistent.

- **Commit:** If the transaction is successful, the changes are permanently applied.
- **Abort/Rollback:** If the transaction fails, any changes made during the transaction are discarded.

Example: Consider the following transaction **T** consisting of **T1** and **T2** : Transfer of \$100 from account **X** to account **Y** .

Before: X : 500	Y : 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Atomicity

Atomicity

If the transaction fails after completion of T1 but before completion of T2, the database would be left in an inconsistent state. With Atomicity, if any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

2. Consistency

Consistency in transactions means that the database must remain in a valid state before and after a transaction.

- A valid state follows all defined rules, constraints, and relationships (like primary keys, foreign keys, etc.).
- If a transaction violates any of these rules, it is rolled back to prevent corrupt or invalid data.
- If a transaction deducts money from one account but doesn't add it to another (in a transfer), it violates consistency.

Example: Suppose the sum of all balances in a bank system should always be constant. Before a transfer, the total balance is \$700. After the transaction, the total balance should remain \$700. If the transaction fails in the middle (like updating one account but not the other), the system should maintain its consistency by rolling back the transaction.

Total before T occurs = $500 + 200 = 700$.

Total after T occurs = $400 + 300 = 700$.

X = 500 Rs		Y = 500 Rs	
T		T''	
Read (X)		Read (X)	
X := X * 100		Read (Y)	
Write (X)		Z := X + Y	
Read (Y)		Write (Z)	
Y := Y - 50			
Write (Y)			

Consistency

3. Isolation

Isolation ensures that transactions run independently without affecting each other. Changes made by one transaction are not visible to others until they are committed.

It ensures that the result of concurrent transactions is the same as if they were run one after another, preventing issues like:

- **Dirty reads:** reading uncommitted data
- **Non-repeatable reads:** data changes between two reads
- **Phantom reads:** new rows appear during a transaction

Isolation

X = 500 Rs		Y = 500 Rs	
T		T''	
Read (X) X := X - 50 Write (X) Read (Y) Y := Y - 50 Write (Y)		Read (X) Read (Y) Z := X + Y Write (Z)	

Isolation

Example: Consider two transactions T and T''.

- X = 500, Y = 500

Explanation:

1. Transaction T:

- T wants to transfer \$50 from X to Y.
- T reads Y (value: 500), deducts \$50 from X (new X = 450), and adds \$50 to Y (new Y = 550).

2. Transaction T'':

- T'' starts and reads X (500) and Y (500).
- It calculates the sum: $500 + 500 = 1000$.
- Meanwhile, values of X and Y change to 450 and 550 respectively.
- So, the correct sum should be $450 + 550 = 1000$.
- Isolation ensures that T'' does not read outdated values while another transaction (T) is still in progress.
- Transactions should be independent, and T'' should access the final values only after T commits.
- This avoids inconsistent results, like the incorrect sum calculated by T''.

4. Durability:

Durability ensures that once a transaction is committed, its changes are permanently saved, even if the system fails. The data is stored in non-volatile memory, so the database can recover to its last committed state without losing data.

Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.

How ACID Properties Impact DBMS Design and Operation

The ACID properties, in totality, provide a mechanism to ensure the correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations, and updates that it makes are durably stored.

1. Data Integrity and Consistency

ACID properties safeguard the data integrity of a DBMS by ensuring that transactions either complete successfully or leave no trace if interrupted. They prevent partial updates from corrupting the data and ensure that the database transitions only between valid states.

2. Concurrency Control

ACID properties provide a solid framework for managing concurrent transactions. Isolation ensures that transactions do not interfere with each other, preventing data anomalies such as lost updates, temporary inconsistency, and uncommitted data.

3. Recovery and Fault Tolerance

Durability ensures that even if a system crashes, the database can recover to a consistent state. Thanks to the Atomicity and Durability properties, if a transaction fails midway, the database remains in a consistent state.

Property	Responsibility for maintaining properties
Atomicity	Transaction Manager
Consistency	Application programmer
Isolation	Concurrency Control Manager
Durability	Recovery

Critical Use Cases for ACID in Databases

In modern applications, ensuring the reliability and consistency of data is crucial. ACID properties are fundamental in sectors like:

- **Banking:** Transactions involving money transfers, deposits, or withdrawals must maintain strict consistency and durability to prevent errors and fraud.
- **E-commerce:** Ensuring that inventory counts, orders, and customer details are handled correctly and consistently, even during high traffic, requires ACID compliance.
- **Healthcare:** Patient records, test results, and prescriptions must adhere to strict consistency, integrity, and security standards.